

Overview

Track 16.6 —  AI on AWS with Amazon Bedrock · 16-ai-tech-stack/tracks/aws-bedrock/README.md

16.6 — AI on AWS with Amazon Bedrock

Where you are: Track 6 of the AI Tech Stack module (16). **Time:** ~10–14 hours · **Prereq:** AWS basics (IAM, VPC, CloudWatch), Python, and Modules 06 [06-ai-agents], 08 [08-rag] for the agent and RAG concepts this track wires into AWS.

Bedrock is not a better model. It is a better *procurement and security story* — an AWS-authenticated front door to models you did not train, with IAM in front, CloudTrail behind, and no API key to rotate. For a two-person startup that is overhead. Inside an enterprise where sending customer data to a third-party SaaS triggers a six-week review, it is the difference between shipping this quarter and shipping next year.

This track teaches the surface as it exists **now** — which means teaching that Bedrock Agents is in maintenance mode, that Converse has replaced per-provider InvokeModel bodies, and that half the Bedrock tutorials on the internet are describing a service that has moved on.

Learning objectives

By the end of this track you can:

1. Decide, with a defensible written rationale, whether a given workload belongs on Bedrock, on a model provider's API directly, or on SageMaker AI.
2. Write the Converse request and response shapes from memory — `system` blocks, `ContentBlocks`, `inferenceConfig`, `stopReason`, `usage` — and explain why building on per-provider `InvokeModel` bodies is a mistake.
3. Implement a complete tool-use loop (`toolConfig` → `stopReason: "tool_use"` → `toolResult` → final turn), including the parallel-tool and error cases, and its streaming equivalent.
4. Choose a Knowledge Base configuration — chunking strategy, vector store, `Retrieve` vs `RetrieveAndGenerate` — and state the conditions under which you would build your own RAG pipeline instead.
5. Write a least-privilege IAM policy for Bedrock that survives cross-Region inference, and enforce a versioned guardrail through an IAM condition.
6. Produce a cost estimate with explicit assumptions, and name the four levers that change it, in the order that actually pays.
7. Handle quotas, throttling, retries with jitter, and multi-Region failover — and explain which Bedrock errors must never be retried.

Suggested path

#	Do this	File	Time
1	Read the core concepts	notes/01-core-concepts.md	2h
2	Run the offline simulator, read every JSON block it prints	code/bedrock_simulator.py	1h
3	Read the annotated boto3 reference	code/converse_api_reference.py	1h
4	Read the production craft note	notes/02-production-craft.md	2h
5	Work through the IAM, guardrail and VPC examples	code/iam_and_guardrails_examples.md	1h
6	Present it back	slides/ (aws-bedrock.pptx)	1h
7	Do the exercises	lab/EXERCISES.md	5h
8	Read the primary docs	papers/PAPERS.md	2h

Both scripts run with **no AWS account, no credentials, and no boto3**:

```
python code/bedrock_simulator.py      # fully offline, always
python code/converse_api_reference.py  # prints wire shapes, exits 0
```

The 16 terms you must own

Converse · ConverseStream · ContentBlock · stopReason · toolConfig · toolUse / toolResult · inference profile · cross-Region inference (CRIS) · model access · guardrail · contextual grounding · Knowledge Base · Retrieve vs RetrieveAndGenerate · cachePoint · Provisioned Throughput · ThrottlingException

Exit check

Hand a colleague a one-page design document for a production RAG assistant on AWS that contains: a reference architecture diagram naming every VPC endpoint; the exact IAM policy JSON, including the cross-Region destination Regions; a cost estimate with its assumptions written down and a link to the pricing page rather than a quoted rate; the guardrail configuration with a justification for each threshold; and a written answer to "what happens when `us-east-1` is throttled?"

If they can find the model ID, the guardrail version, and the failover plan without asking you a question, you passed.